

Zbots: Simulation Porting, FARMSIM Integration, and Fluid Dynamics for CPG-Driven Bout-and-Glide Swimming

Joshua John Leo Valan Johnson

Roll No: 110123044

National Institute of Technology Tiruchirappalli

Work carried out at the Biorobotics Laboratory (BIOROB),
Institute of Bioengineering, École Polytechnique Fédérale de Lausanne (EPFL)

Under the guidance of

M. Louis Gevers
PhD scholar, BIOROB
EPFL

Prof. Auke Ijspeert
Professor and Head, BIOROB
EPFL

August 2026

Abstract

ZBot is an ~ 80 cm fishlike robot at EPFL's Biorobotics Laboratory, built to compare intermittent bout-and-glide swimming against continuous undulation. This report covers the simulation and control infrastructure built for it during a nine-week internship: the robot's Webots model ported to MuJoCo, a neuromechanical simulation assembled from scratch on FARMS, and a CPG-driven bout-and-glide controller mirroring the robot's C++ firmware.

Porting the model exposed defects in the physics beneath it, and correcting them became the larger part of the work. Water density was hard-coded into the buoyancy computation; making it configurable is a small fix, now merged upstream. More consequentially, buoyancy was applied at a fixed proxy point rather than at the centre of buoyancy, so a body simulated that way floats at the right depth and never rights itself. I replaced this with a tetrahedron decomposition that computes the true centre of buoyancy from each body's own collision geometry. It matches closed-form settling theory to a fraction of a millimetre and, in the test that distinguishes it from the model it replaces, reproduces classical metacentric stability, correctly predicting the densities at which a floating cube capsizes.

The exact computation is too expensive to run every physics step on this geometry, so the delivered default decimates it to one step in twenty and interpolates in between, which keeps the simulation several times faster than real time at the cost of a few tens of milliseconds of lag in the buoyancy force. That trade-off, and the other limitations the work leaves open, are set out at the end.

Contents

Nomenclature	4
1 Introduction	5
1.1 Internship Context	5
1.2 The Robot	5
1.3 Objectives and Mandate	5
1.4 Scope and Structure of This Report	6
2 Simulation Port: Webots to MuJoCo	6
2.1 Motivation for the Port	6
2.2 Scope of the Port	6
2.3 Coordinate Frames, Geometry, and Inertia at Port Time	7
2.4 Validation: Reproducing a Traveling Wave	7
3 FARMSIM Integration with the Animat Controller	8
3.1 Overview of the FARMSIM Framework	8
3.2 From-Scratch Implementation on the MuJoCo Port	8
3.3 Animat Controller Integration	9
3.4 Containerized, Cross-Platform Deployment	9
4 Bout-and-Glide Locomotion via CPG	10
4.1 CPG Oscillator Model	10
4.1.1 The Fully-Coupled Reference Model	10
4.1.2 Structural Simplification of the ZBot Implementation	10
4.2 Bout Gate, vSPN, and Motor Neurons	10
4.2.1 Motor Neuron Layer	11
4.3 The Bout-and-Glide State Machine	12
4.4 Control-Rate Sensitivity of the Bout Gate	12
4.5 Rationale for Bout-and-Glide over Continuous Undulation	13
5 Upstream Contribution to farms_mujoco: Configurable Water Density	14
5.1 Upstream Repository and Change	14
5.2 Buoyancy Formula Before and After the Fix	14
6 Tetrahedron-Method Buoyancy: Centre of Buoyancy from Collision Geometry	15
6.1 The Problem: Coincident Centre of Buoyancy and Centre of Mass	15
6.2 The Fix: Per-Primitive Tetrahedron Decomposition of Submerged Volume	15
6.2.1 Stage 1: From Primitive to Mesh	15
6.2.2 Stage 2: Tetrahedron Decomposition of the Submerged Region	16
6.2.3 Stage 3: Signed Volume, Weighted Centroid, and the Restored Torque	17
6.2.4 The Winding-Order Convention	18
6.2.5 Correctness of the Common-Apex Fan Construction	19
6.2.6 Mesh Discretization Error	20
6.2.7 Closed-Form Shapes	20
6.3 Validation Against Closed-Form Settling Theory	21
6.3.1 Steady-State Error	21
6.4 Validation of the Righting Torque Against Metacentric Theory	22
6.5 The Method on the Robot Itself	24
6.6 Cost, and How It Is Kept Inside the Real-Time Budget	24

6.6.1	The Four Selectable Methods	25
6.6.2	Measured Cost	25
7	Deliverables and Current Status	26
7.1	Delivered Artifacts	26
7.2	Scope Boundary and Next Steps	26
8	Known Issues and Future Work	27
8.1	The Inertia Tensor Problem	27
8.1.1	Recovering the Principal Frame	27
8.2	Why the Current Model Is Still Wrong	28
8.3	The Case for System Identification	28
8.4	Future Work	29
9	Conclusion	29
	Acknowledgments	30

Nomenclature

CPG oscillators and gating (Section 4)	
$\theta_i, x_i^{\text{CPG}}$	phase and output of oscillator i
f	tail-beat (CPG) frequency
n	number of body segments ($n = 6$); $2n$ oscillators total
$\Delta\phi_{\text{seg}}$	rostrocaudal phase lag between segments
w_{ij}, ϕ_{ij}, ν_i	coupling weight, phase bias and intrinsic frequency of the reference model
$E(\tau), V$	raw exponential envelope, IIR-smoothed envelope
A_0, α, β	initial amplitude, per-second decay base, IIR smoothing weight
τ	time elapsed since the last SWIMMING transition
T_d	continuous-time decay constant of $E(\tau)$, $T_d = -1/\ln \alpha$
$T_\beta, \Delta t$	effective time constant of the discrete IIR filter, $T_\beta = \Delta t/(1 - \beta)$; control period
$L, L_{\text{th}}, \dot{d}$	leaky integrator value, threshold, drive rate
κ	gliding-amplitude threshold fraction
$T_{\text{bout}}, T_{\text{glide}}$	target bout and glide durations
m_i, ω, b	motor-neuron activation, sigmoid gain, sigmoid bias
x_s, k_T, g_s, c	motor command for segment s , tail-amplitude ratio, per-segment bias, OMR gain
Tetrahedron buoyancy method (Section 6)	
ρ, ρ_w	link material density, water density
m, g	link mass, gravitational acceleration ($g = -9.81 \text{ m/s}^2$, z up)
h_s, r, z	water-surface height, link bounding radius, link vertical position
$\vec{r}_{\text{cob}}, \vec{r}_{\text{com}}$	centre of buoyancy, centre of mass
$\vec{F}_b, \vec{\tau}_b$	buoyancy force, buoyancy torque
$\vec{p}, \hat{n}$	tetrahedron-fan apex point, outward unit face normal
V_Δ, V	signed tetrahedron volume, total submerged volume
$n_{\text{lat}}, n_{\text{lon}}, n_{\text{seg}}$	mesh latitude rings, longitude vertices, cylinder segments
V_{sub}, Z	submerged volume, settled centre height relative to the free surface
GM, I_{wp}	metacentric height, waterplane second moment
Inertia (Section 8.1)	
$\mathbf{I}, I_1, I_2, I_3$	inertia tensor, principal moments
θ	tilt of the principal frame off the body frame

1 Introduction

1.1 Internship Context

This report documents a nine-week research internship at the Biorobotics Laboratory (BIOROB), Institute of Bioengineering, École Polytechnique Fédérale de Lausanne (EPFL), running from June 23 to August 28, 2026, under the supervision of M. Louis Gevers (PhD scholar, BIOROB) and Prof. Auke Ijspeert (BIOROB). The work contributed to the laboratory’s ZBot project, an ongoing investigation into the energetics of intermittent swimming in a fishlike robot.

1.2 The Robot

ZBot is an undulatory swimming robot developed at BIOROB. It serves as a testing platform for comparing the energy efficiency of bout-and-glide (intermittent) swimming against continuous undulation in a fishlike robot [2]. ZBot is a bioinspired scale model of the larval zebrafish (*Danio rerio*), magnified roughly 200-fold: a ~ 4 mm larva becomes an ~ 80 cm robot. This scale lets the mechanism carry real actuators, a power meter, and onboard sensors that would not fit inside a live larva.

Physically, ZBot has a head segment housing its electronics: controller, camera, remote-control receiver, battery, wattmeter. A chain of six serially linked, actuated body segments follows, ending in a caudal fin. This gives six controllable degrees of freedom along the trunk and tail [2]. The MuJoCo model I built reproduces this exactly: a **Head** body, six **Segment** bodies, and a passive **TailSegment**, each connected by a single-axis hinge joint (Section 2). The onboard camera and remote-control link also let researchers pilot the robot directly in open water and in controlled pool experiments [2].

A spinal central pattern generator (CPG) drives locomotion: a network of coupled oscillators that produces rhythmic, left/right-alternating bending commands along the body, without computing an explicit trajectory at every instant. Gating its output makes the robot alternate between active tail-beating and passive coasting. This bout-and-glide pattern is the central locomotion strategy of this project. Section 4 derives the full CPG/bout-gate mechanism.

1.3 Objectives and Mandate

The mandate I was given at the start of the internship was an infrastructure one. MuJoCo is the laboratory’s standard physics simulator, and FARMS (Section 3) is the neuromechanical framework the laboratory builds on top of it; the ZBot project, however, still ran entirely in Webots. The assigned objective was therefore to bring ZBot onto that standard stack: first port the robot model and simulation from Webots to MuJoCo, then integrate the result with FARMS so that the robot could be simulated, controlled, and logged with the same tooling as the laboratory’s other animats. Reimplementing the physical robot’s C++ bout-and-glide controller inside that new integration followed directly from this, since a ported model is only useful if it can be driven by the same control law as the hardware.

A second, unplanned body of work emerged from carrying out the first. Porting the model exposed three defects in the simulation physics that had to be addressed before the ported robot behaved correctly in water:

1. the water density in the `farms_mujoco` buoyancy computation was hard-coded, so it could not be configured per experiment (Section 5);
2. the fluid-dynamics code applied the buoyant force at a fixed proxy point (the link’s bounding-sphere centroid, which MuJoCo places at the centre of mass) rather than at the centre of buoyancy, so it produces no righting torque at all (Section 6);
3. the robot’s inertia tensor was not expressed in its own principal-axis frame (Section 8).

The first I fixed and merged upstream. The second I replaced with a geometrically correct method, which runs inside the real-time budget only because the exact computation is decimated (Section 6.6). The third I diagnosed but did not close. This split between the assigned porting/integration objective and the emergent physics work is the main reason the report devotes as much space as it does to buoyancy.

1.4 Scope and Structure of This Report

The Webots simulation and the C++ hardware controller both pre-date this project; everything downstream of them is my own work. I built the MuJoCo model and simulation port from the existing Webots description. No FARMS integration existed for ZBot before this project, so I built one from scratch on top of the MuJoCo port. I then built the CPG-driven bout-and-glide controller, the upstream `farms_mujoco` fix, and the buoyancy model inside that new FARMS integration. This report follows the same order: simulation port (Section 2), FARMS integration (Section 3), the CPG controller (Section 4), the two physics contributions (Sections 5–6), delivered artifacts and current status (Section 7), and known issues and future work (Section 8), which covers the outstanding inertia-tensor problem and the directions that follow from here. Section 9 concludes.

Throughout, reported numbers were measured by running the delivered modules rather than quoted from elsewhere; where a claim rests on something I could not verify, the text says so.

2 Simulation Port: Webots to MuJoCo

2.1 Motivation for the Port

Before this project, the ZBot simulation existed only in Webots. It used a custom robot description with a native fluid-dynamics model: a per-segment drag force and torque, with separate coefficients for the head, body, and tail segments.

The Webots fluid model was the immediate trigger for the move. Its drag and buoyancy behaviour is a Webots-native plugin, existing only inside Webots’ own physics loop and coupled to its ODE-based rigid-body solver — and the FARMS paper identifies ODE as prone to instability for models with many joints, because it solves dynamics in maximal coordinates [1].

MuJoCo [9] is the lab’s standard simulator, and two practical reasons beyond raw physics made it the destination. Its model format, MJCF, is plain XML with a documented schema and imports from URDF, which is the route this port took — though not without two hand-applied corrective rotations; Webots’ PROTO format is not portable outside Webots at all. And MuJoCo is scriptable from Python, which matters because the lab’s own tooling, including FARMSIM (Section 3), is Python-based, whereas the existing ZBot Webots controller was written in C++.

2.2 Scope of the Port

Starting from the existing Webots description, I built the MuJoCo model and the surrounding simulation code from scratch:

- **Model description:** the primary MJCF model, an intermediate non-reoriented version, and the URDF it was converted from. A header comment records two hand-applied rotations on top of the raw URDF-to-MJCF conversion: an Euler rotation correcting the collision bounding boxes, and a +90° rotation about z to place the robot on the arena floor. Both are evidence of the coordinate-frame reconciliation a cross-simulator port needs.
- **Robot interface:** a thin `RobotModel` wrapper around MuJoCo’s model and data objects. It resolves the six joint names and their position- and torque-actuator IDs, and exposes joint position/velocity getters and command setters.

- **Simulation loop:** a `Simulator` class that splits the physics step into a kinematics/dynamics pass and a state-integration pass, with a hook in between for an environment object to apply external forces. This hook is the seam where I later inject fluid forces, either through a native MuJoCo plugin or left absent for a dry run.
- **Controllers:** an abstract `BaseController` interface, an `OpenLoopController` that replays a fixed target-position vector, and a `CPGController` implementing the traveling-wave sanity check from the validation step below.
- **Environments:** `DryEnvironment` and `PoolEnvironment`. Both are trivial in the port itself: I implemented the actual hydrodynamics later, at the FARMSIM level (Section 3), not in this lightweight wrapper.
- **Run scripts,** driving the whole loop end to end.

The robot model itself is a free-floating `Head` body (with a `freejoint`), followed by a chain of six hinge-jointed `Segment` bodies (`joint_1...joint_6`, axis $(0, 1, 0)$, range $\approx \pm 1.13$ rad), and a passive `TailSegment`. This matches the six actuated body segments reported for the physical robot [2]. Each hinge joint carries both a position actuator (`pos_motor_i`, $k_p = 1.0$) and a torque actuator (`torq_motor_i`, ± 3.0 N·m), so the same model runs under either position or torque control, depending on the controller.

2.3 Coordinate Frames, Geometry, and Inertia at Port Time

I kept collision geometry deliberately simple relative to the visual meshes. Every body carries a non-colliding, render-only visual mesh, alongside a small set of primitive collision geometries (cylinders and boxes) sized to approximate the same volume. The tetrahedron buoyancy method of Section 6 consumes these same primitives.

At port time, each body’s inertia was a diagonal tensor together with a rotation carrying the inertial frame into the body’s principal axes. This correctly captured that a body’s principal axes need not coincide with its own geometric axes. Two later passes over the model rewrote that representation, and the second of them dropped the rotation. Section 8 traces both.

2.4 Validation: Reproducing a Traveling Wave

As a first sanity check, once the coordinate frames and joint definitions were transferred, I validated the ported model by driving all six joints with a simple, explicit traveling-wave signal, rather than the full CPG network. The validation controller computes, for joint $i \in \{0, \dots, 5\}$ and time t ,

$$\theta_i(t) = A \sin(2\pi f t - i \Delta\phi), \quad (1)$$

where A is a fixed amplitude, f the beat frequency, and $\Delta\phi$ a constant phase lag applied per joint index along the body (default $A = 0.5$, $f = 1.0$ Hz, $\Delta\phi = \pi/3$ in the code). This is a minimal open-loop probe signal, not the neural model; the CPG network comes later, in Section 4. Figure 1 shows the ported model holding the resulting body wave.

Eq. (1) is the kinematic pattern the CPG-driven controller is later expected to produce: a sinusoidal bending command whose phase decreases monotonically from head to tail. Reproducing it with the correct handedness, and without unexpected coupling between joints, shows that the rigid-body chain, joint axes and actuator wiring survived the port.

A second, separate experiment (`zbot_swimming`) goes further, configuring a full-body CPG network rather than an imposed waveform. It declares one left/right oscillator pair per segment with the same contralateral and rostrocaudal phase biases. This targets the same rostrocaudal phase-lag pattern as the structurally-fixed network of Section 4, but reaches it as the fixed point of a dynamically-coupled network in the form of Eq. (2), rather than by freezing the phase differences at reset as the bout-and-glide controller does.

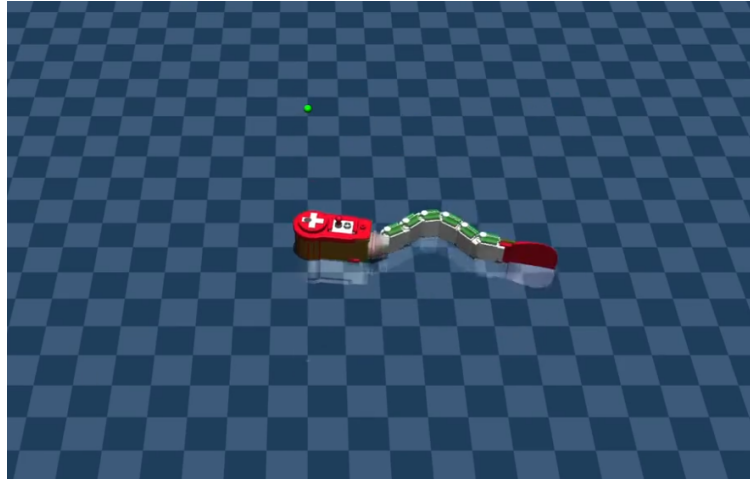


Figure 1: The ported MuJoCo model under the traveling-wave validation signal of Eq. (1): head segment, six actuated **Segment** bodies and the passive **TailSegment**, holding the rostro-caudal body wave that the port is required to reproduce.

3 FARMSIM Integration with the Animat Controller

3.1 Overview of the FARMSIM Framework

FARMS (Framework for Animal and Robot Modeling and Simulation) is BIOROB’s open-source, interdisciplinary Python framework for neuromechanical simulation [1]. Its stated goal is to unify model description, physics simulation, and result analysis for both animals and robots inside one modular pipeline. Musculoskeletal models, neural-network controllers, sensors, and physics backends can then be mixed and matched, without re-implementing simulation infrastructure for every new study.

Architecturally, FARMS splits a simulation into three stages over a shared, standardized data layer of configuration files, run-time data and processed results: *modeling* describes the animat’s rig (links, joints, inertial elements, geometry, actuators) and the environment’s terrain and fluid properties; *simulation* runs each control tick through five sub-stages — sensors, external forces, controller, actuators, physics step; and *analysis* post-processes, visualizes and replays the logged run [1]. Bullet and MuJoCo were originally interchangeable backends, but `farms_mujoco` is the actively maintained one, the Bullet path having been retired in practice.

3.2 From-Scratch Implementation on the MuJoCo Port

I did not adopt an existing FARMSIM deployment: FARMS had no ZBot integration to adopt. I built the swimming/hydrodynamics layer, the ZBot animat controller extension, and the surrounding experiment scaffolding from scratch, on top of the MuJoCo port from Section 2, on a tree vendoring `farms_core`, `farms_mujoco`, `farms_amphibious`, and `farms_sim` as git submodules. Concretely, this meant:

- Writing the animat description (SDF-based robot and arena models) that FARMS’ modeling stage consumes, derived from the same physical robot geometry as the MJCF port.
- Implementing the `ZBotCPGController` extension (Section 4) against FARMS’ controller interface, so the C++ hardware controller’s logic runs inside the FARMS simulation loop, not the lightweight simulator from the initial port validation.
- Extending the swimming subsystem with the tetrahedron-based buoyancy method and its real-time optimizations (Section 6). Neither existed in the upstream framework; both

now sit alongside the drag/hydrodynamics code as project-specific, upstreamable work (Section 5 covers the water-density contribution I merged upstream).

- Building the experiment scaffolding, configuration files, and validation harness needed to configure, run, and analyze FARMSIM experiments for this robot.

The payoff for building on the lab-standard framework rather than extending the bespoke loop of Section 2 is that the controller and hydrodynamics work inherits FARMS’ standardized data logging and five-stage simulation loop for free, and the ZBot simulation becomes directly comparable to, and reusable by, the lab’s other animat models [1].

3.3 Animat Controller Integration

FARMS registers the animat controller through the extension mechanism exposed by `AnimatController`. I declare the ZBot controller, `ZBotCPGController`, in the animat’s YAML configuration as an `extensions` entry:

```
extensions:
- loader: controller.zbot_controller.ZBotCPGController
  config:
    swimming_mode: bout_and_glide
    bout_duration_s: 5.0
    bout_interval_s: 2.0
    tail_frequency: 1.0
    tail_amplitude: 1.0
```

FARMS instantiates it through a `from_options` classmethod, which reads the joint names and control types from the animat’s motor configuration, builds the per-joint torque limits, and constructs the controller’s parameter object: directly, or, when a bout duration and interval are given, through a `from_bout_timing` convenience constructor (Section 4). At every simulation step, FARMS calls the controller’s `before_step` hook, which advances the CPG/bout-gate/vSPN state machine by one timestep. It then reads the resulting joint targets back through a `positions` call, which FARMS’ PD position-control layer applies to the six actuators. The controller is explicitly open-loop: I leave the sensory-feedback arguments FARMS passes in unused, since every joint command comes purely from the controller’s own internal dynamics.

The rate at which `before_step` is called is not fixed by the controller itself. It follows from the experiment’s physics configuration: a 2 ms timestep with `cb_sub_steps: 2`, so the hook fires twice per environment step, at 1 kHz. That is twenty times the C++ hardware controller’s own 20 ms default period, and Section 4.4 works through what that implies for the bout-gate filter.

3.4 Containerized, Cross-Platform Deployment

The FARMSIM deployment is containerized, with a Linux and a Windows configuration whose Dockerfiles are identical, so both build the same stack — a pinned Python version, the toolchain for `farms_mujoco`’s Cython extensions, and the same four FARMS submodules — with no host-specific divergence in the image. Their Compose files differ in the expected ways — display forwarding and SSH-key paths — and in one that is worth knowing: the Linux configuration bind-mounts only `farms/farms_mujoco`, while the Windows one mounts the whole `farms/` tree. Edits to `farms_core`, `farms_sim` or `farms_amphibious` are therefore live on Windows but not on Linux, where those come from whatever was baked into the image. The asymmetry looks unintentional, and the two mount lists should be reconciled.

4 Bout-and-Glide Locomotion via CPG

4.1 CPG Oscillator Model

4.1.1 The Fully-Coupled Reference Model

My oscillator network descends from the coupled-oscillator spinal-cord model of Ijspeert et al. [3], developed originally for a salamander robot, reviewed more generally by Ijspeert [4], and widely reused across the lab’s later CPG-driven robots. In that model each oscillator i carries a phase θ_i and an amplitude r_i , and couples to every other oscillator j through a coupling weight w_{ij} and phase bias ϕ_{ij} . The phase dynamics are what matter for the comparison below:

$$\dot{\theta}_i = 2\pi\nu_i + \sum_j r_j w_{ij} \sin(\theta_j - \theta_i - \phi_{ij}), \quad (2)$$

with ν_i the oscillator’s intrinsic frequency, set by a descending drive. A positive coupling weight w_{ij} actively pulls oscillator i ’s phase toward $\theta_j - \phi_{ij}$, the fixed point of the coupling term, so a traveling body wave emerges from the network dynamically, as the fixed point of the coupled system, rather than being imposed externally. The model’s companion amplitude dynamics and burst-output mapping [3] are not reproduced here: the ZBot controller replaces them with the exponential bout envelope of Section 4.2.

4.1.2 Structural Simplification of the ZBot Implementation

The C++ hardware controller is the canonical implementation. Its oscillator, filter, leaky-integrator and top-level classes are mirrored class-for-class by the Python FARMSIM extension `ZBotCPGController`; the equations below use the Python notation but are identical, term for term, to the C++ source. That equivalence was established by reading both implementations side by side, not by comparing against logged hardware traces, which were not available; it is a statement about the code rather than a measured agreement in behaviour.

The network has $2n$ phase oscillators for $n = 6$ segments, one left/right pair each. Every oscillator shares a single intrinsic frequency f , and its phase integrates as

$$\theta_i(t + \Delta t) = (\theta_i(t) + 2\pi f \Delta t) \bmod 2\pi, \quad i = 0, \dots, 2n - 1, \quad (3)$$

with output $x_i^{\text{CPG}} = \sin \theta_i + 1 \in [0, 2]$. Unlike Eq. (2), the coupling here is *structural* rather than dynamically integrated: the phase relationships are fixed once at reset,

$$\theta_{\text{left},s}(0) = -s \Delta\phi_{\text{seg}}, \quad (4)$$

$$\theta_{\text{right},s}(0) = -s \Delta\phi_{\text{seg}} + \pi, \quad s = 0, \dots, 5, \quad (5)$$

with $\Delta\phi_{\text{seg}} = 1.2\pi/6 \approx 0.628$ rad and contralateral antiphase π . Since every oscillator advances at the same rate, these differences never drift, and the network produces a head-to-tail wave of fixed wavelength without needing an explicit coupling term. This is a deliberate simplification suited to a single fixed gait on embedded hardware with a hard real-time budget; it costs the flexibility — per-oscillator frequency modulation, runtime-adjustable coupling — that the fully-coupled model offers.

4.2 Bout Gate, vSPN, and Motor Neurons

The CPG’s amplitude is not constant. An *exponential filter* shapes it, matching the “low-pass-filtered exponential function” description of the bout gate and vSPN (ventral spinal projection neuron) envelopes reported for the physical robot’s neural model [2]. Let τ be the time elapsed since the controller’s last transition into SWIMMING. Given an initial amplitude A_0 , a per-second decay base $\alpha \in (0, 1)$, and an IIR smoothing weight β , the filter’s two state variables

are

$$E(\tau) = A_0 \alpha^\tau, \quad (6)$$

$$V_{k+1} = \beta V_k + (1 - \beta) E(\tau_k), \quad (7)$$

where $E(\tau)$ is the raw decaying exponential, evaluated at the current elapsed time; V_k is the IIR-smoothed value actually used as the CPG amplitude, updated once per control step k from the current sample $E(\tau_k)$. Eq. (6) is equivalently a continuous-time exponential decay,

$$E(\tau) = A_0 e^{-\tau/T_d}, \quad T_d = -\frac{1}{\ln \alpha}, \quad (8)$$

where T_d is the time for the envelope to decay by a factor of e — a more physically meaningful quantity than α itself, and independent of the control rate. At the default $\alpha = 0.63$ this is about 2.2 s. Two instances of this filter run in parallel, both with $A_0 = 1.25$: the *bout gate* shapes the CPG amplitude, and the *vSPN* shapes the turning drive used during a turning bout.

A separate *leaky integrator* accumulates a descending drive $\dot{d}$ (units/s) during gliding,

$$L(t + \Delta t) = L(t) + \dot{d} \Delta t, \quad (9)$$

where L is the accumulated value: the controller switches from gliding back to swimming once $L \geq L_{\text{th}}$. This is the biologically-observed mechanism whereby a supraspinal gating centre re-activates the CPG when accumulated descending input crosses a threshold [2]. Despite the name inherited from that concept, the integrator has no leak term — it is purely additive, as both the C++ and Python implementations confirm — and because it scales its increment by Δt explicitly, unlike Eq. (7), it does not carry the control-rate sensitivity of Section 4.4.

4.2.1 Motor Neuron Layer

A sigmoid layer converts the twelve oscillator outputs to motor-neuron activations. Index the oscillators $i = 0, \dots, 11$, with side given by parity, and let $\text{turn} \in \{\text{LEFT}, \text{RIGHT}, \text{NONE}\}$ be the commanded turning side. Each activation is

$$m_i = \sigma(\omega (A_{\text{cpg}} x_i^{\text{cpg}} + A_{\text{vspn}} \mathbf{1}[\text{side}(i) \neq \text{turn}] - b)), \quad \sigma(z) = \frac{1}{1 + e^{-z}}, \quad (10)$$

with gain $\omega = 1.4$ and bias $b = 2.0$. A_{cpg} and A_{vspn} are the bout-gate and vSPN filter values, both zero while gliding.

The indicator routes the vSPN turning drive to the side *opposite* the commanded one, which looks backwards but is not: a commanded left turn raises the right-side activations, drives the left/right difference negative, and bends the body toward negative joint angles. Evaluating the compiled model confirms that negative is indeed left. In the spawn pose world-up maps onto body $+y$ and the body runs head-to-tail along $+z$, so forward is $-z$ and left = up $\times$ forward = $\hat{y} \times (-\hat{z}) = -\hat{x}$. The hinges turn about $+y$, so a positive command displaces the tail toward $+x$ and a negative one toward $-x$ — which is left, as commanded.

The final motor command for segment s is the left/right difference, scaled by a per-segment gain and a constant carried over from the original optomotor-response implementation:

$$x_s = k_T g_s (m_{\text{left},s} - m_{\text{right},s}) c, \quad s = 0, \dots, 5, \quad (11)$$

with a tail-amplitude ratio $k_T = 1.0$ and a constant $c = 1.5^2 = 2.25$ inherited from that implementation. The six per-segment biases increase head to tail,

$$g_0, \dots, g_5 = 0.203, 0.364, 0.481, 0.594, 0.696, 1.0, \quad (12)$$

giving a growing wave envelope. Eq. (11) is the servomotor-output relation reported for the robot [2].

At the 1 kHz control rate the caudal joint's peak command reaches 1.208 rad (69.2°), slightly past its ± 1.13 rad range, so the position actuator clips for a small fraction of each cycle. Section 4.4 shows this is a consequence of the control rate rather than of the controller: at the hardware's own period the peak is 31.3° and nothing clips.

4.3 The Bout-and-Glide State Machine

The controller is a two-state machine, SWIMMING and GLIDING:

- **Bout (SWIMMING):** the bout gate and vSPN filters advance (Eqs. (6)–(7)), and so do the CPG phases (Eq. (3)). The body actively undulates under CPG drive, and the amplitude decays exponentially from its onset value.
- **Glide (GLIDING):** the CPG freezes, its amplitude forced to zero ($A_{\text{cpg}} = 0$ in Eq. (10)). The left/right motor-neuron difference collapses toward zero, and the joints relax toward the neutral posture. Only the leaky integrator (Eq. (9)) keeps advancing. Physically, the robot coasts ballistically along the impulse vector from the preceding bout, unable to steer: exactly the “glide” phase of larval zebrafish burst-and-coast swimming [2].

The transition equations governing the gate between the two phases are

$$\text{SWIMMING} \rightarrow \text{GLIDING} \quad \text{when} \quad E(\tau) < \kappa A_0, \quad (13)$$

$$\text{GLIDING} \rightarrow \text{SWIMMING} \quad \text{when} \quad L(t) \geq L_{\text{th}}, \quad (14)$$

Eq. (13) tests the bout gate’s *raw* exponential $E(\tau)$ (Eq. (6)), not the IIR-smoothed V : this keeps the bout duration deterministic, independent of the smoothing dynamics and hence of the control-rate sensitivity described in Section 4.4. $\kappa = 0.1$ is the gliding-amplitude threshold fraction. On each transition, the bout gate, vSPN, and CPG phases all reset to their initial state (Eq. (5)), so every bout starts from the same posture and amplitude ramp.

A convenience constructor converts an intuitive (bout duration, bout interval) pair to the low-level decay rate and drive rate. It solves $A_0 \alpha^{T_{\text{bout}}} = \kappa A_0$ for α , and sets the drive rate to reach L_{th} at T_{glide} :

$$\alpha = \kappa^{1/T_{\text{bout}}}, \quad \dot{d} = \frac{L_{\text{th}}}{T_{\text{glide}}}, \quad (15)$$

so the two intuitive durations set the decay and drive rates directly. The default configuration asks for a 5 s bout and a 2 s glide.

Figure 2 shows the resulting E , V , L , and state trajectory across three consecutive bouts, computed directly from Eqs. (6)–(9) at the 1 kHz control rate with these default parameters.

4.4 Control-Rate Sensitivity of the Bout Gate

One property of the filter is worth recording, because it is invisible in the parameter list. Unlike $E(\tau)$, which depends only on elapsed time, the recursion $V_{k+1} = \beta V_k + (1 - \beta)E(\tau_k)$ applies β once per *call*, with no reference to Δt . Its effective time constant is therefore

$$T_\beta = \frac{\Delta t}{1 - \beta}, \quad (16)$$

so at $\beta = 0.975$ the gate’s smoothing is set entirely by how often it is called: $T_\beta \approx 0.8$ s at the hardware’s 20 ms period, and ≈ 40 ms at the 1 kHz rate the FARMS experiment runs. The leaky integrator, by contrast, scales its increment by Δt explicitly and is unaffected.

Because V restarts from zero at each bout onset while E is already decaying, how fast V rises decides how much of the envelope it captures, so the difference shows up in amplitude rather than timing. Integrating the delivered controller’s own classes at both rates gives a peak V of 0.698 at 20 ms against 1.160 at 1 ms, and a peak caudal command of 31.3° against 69.2° (Figure 3). Bout and glide durations are identical at both rates, since the glide test of Eq. (13) reads E rather than V .

The parameters here are the hardware defaults, carried across deliberately and used as-is for the simulation work; the point of this section is not that they are wrong but that β is a per-call

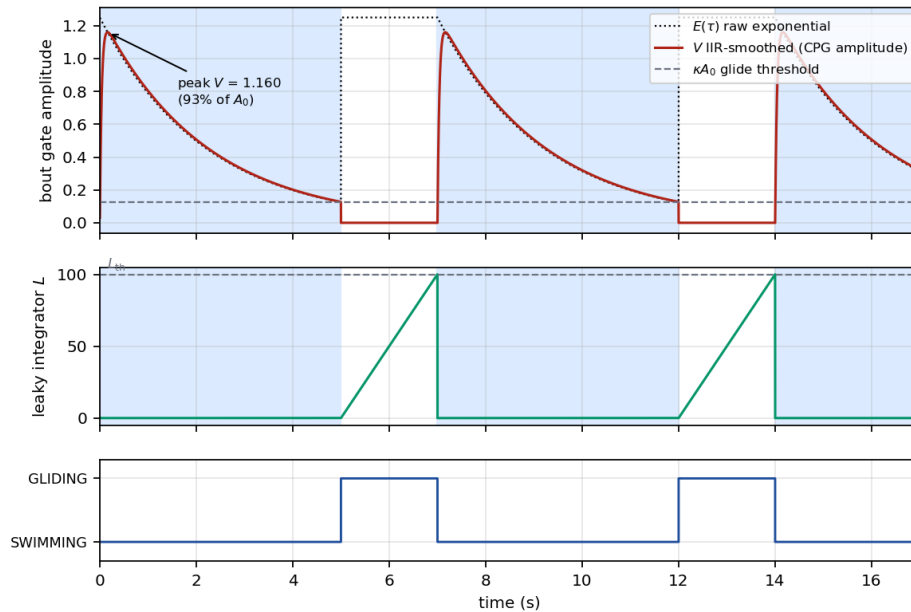


Figure 2: Bout gate state variables at the default parameters ($T_{\text{bout}} = 5.0$ s, $T_{\text{glide}} = 2.0$ s, $L_{\text{th}} = 100$, $\Delta t = 1$ ms), produced by integrating the delivered controller’s own **ExponentialFilter**, **LeakyIntegrator** and **SegmentalCPG** classes rather than by re-implementing Eqs. (6)–(9). Top: raw exponential $E(\tau)$ and IIR-smoothed V against the κA_0 glide threshold, with swimming phases shaded; V restarts from zero at every bout onset while E is already decaying, so it only ever reaches 93% of A_0 . Middle: leaky integrator L against L_{th} , advancing only while gliding. Bottom: the resulting state, giving the configured 5 s bout and 2 s glide. This is an integration of the controller alone, not a MuJoCo/FARMS simulation run.

weight rather than a physical time constant, so the same value does not describe the same filter at a different rate. Anyone comparing simulated tail amplitude against the physical robot should either match the control period or rescale β as $1 - \Delta t/T_\beta$, which at 1 kHz and the hardware’s $T_\beta = 0.8$ s gives $\beta = 0.99875$; the dashed trace in Figure 3 shows that value reproducing the hardware envelope at the simulation’s rate.

4.5 Rationale for Bout-and-Glide over Continuous Undulation

Bout-and-glide (burst-and-coast) swimming is the intermittent locomotion strategy actually observed in larval zebrafish and in adult fish at high speed [5, 6]. It is the specific behaviour ZBot was built to replicate and study mechanistically [2]. The controller supports a **CONTINUOUS** mode as an explicit alternative (constant CPG amplitude, no bout gate, no leaky integrator, turning drive disabled) precisely to compare the two regimes. Liu et al. [2] report that bout-and-glide swimming achieves a lower cost of transport (CoT) than continuous tail-beating over most of ZBot’s achievable velocity range, below ~ 0.32 body-lengths/s in water, across both turbulent (water) and viscous flow regimes. They trace this advantage not primarily to reduced glide drag from a straight-tail posture (the “fluid dynamics hypothesis” originating with Weihs [5]) but to an *actuator efficiency* effect: intermittent locomotion lets the servomotors spend more time in their higher-efficiency payload regime. This holds, they verify, even in viscous liquids, where gliding distance and hence any drag-reduction benefit is negligible. It is that actuator-efficiency argument, rather than a purely fluid-dynamic one, which the controller implemented here reproduces in simulation.

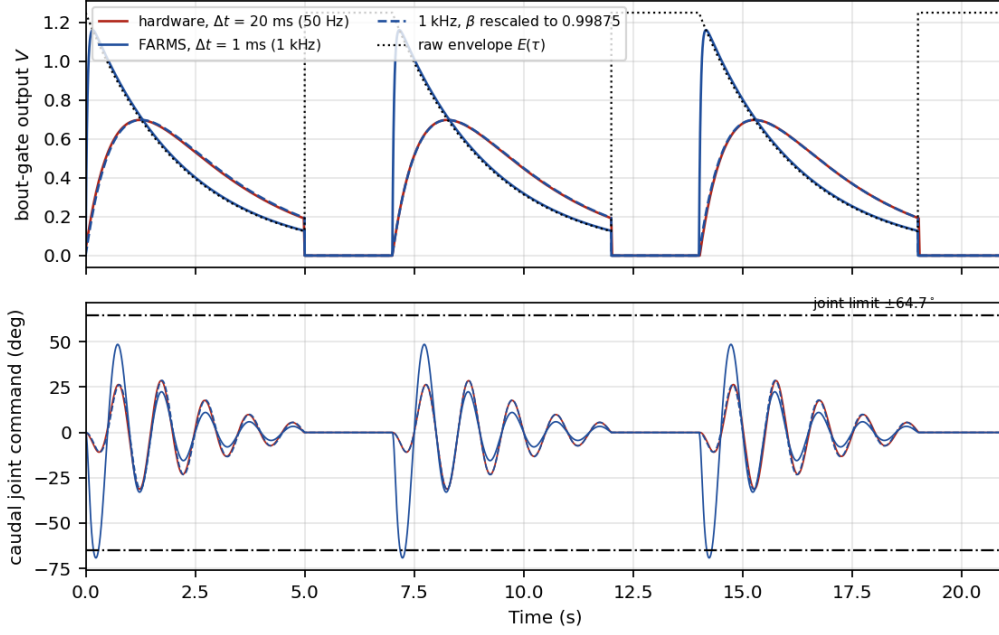


Figure 3: The IIR smoothing weight β is a per-call weight, not a time constant. Top: the bout-gate output V against the raw envelope $E(\tau)$. Bottom: the resulting caudal joint command. At the 1 kHz FARMS rate (solid blue) V tracks E far more tightly than at the hardware’s 50 Hz (solid red), raising the peak command from 31.3° to 69.2° . Rescaling β to 0.99875 (dashed) reproduces the hardware behaviour at the simulation’s rate. Traces come from integrating the delivered controller’s own filter, integrator and CPG classes.

5 Upstream Contribution to farms_mujoco: Configurable Water Density

5.1 Upstream Repository and Change

Previously, the `farms_mujoco` physics backend hard-coded the density of fresh water (1000 kg/m^3) into the buoyancy calculation: changing `water_density` in an experiment’s `animat_config.yaml` had no effect. I fixed this against https://github.com/farmsim/farms_mujoco/tree/amphibious_v0.2 and submitted the fix upstream, where it was merged as pull request #43.

5.2 Buoyancy Formula Before and After the Fix

Before the fix, the swimming extension computed buoyancy force as

$$\vec{F}_b = \left(0, 0, -1000 \frac{mg}{\rho} \min\left(\frac{h_s + r - z}{2r}, 1\right) \right), \quad (17)$$

Here m is the link mass, g the gravitational acceleration as a signed scalar with z pointing up ($g = -9.81 \text{ m/s}^2$, so that $-g$ is positive and the force points upward), ρ the link’s own material density, z the link’s vertical position, r its bounding radius (confusingly named a height in the original code), and h_s the water-surface height. The ramp term $\min(\cdot, 1)$ linearly ramps the force from zero to full buoyancy as the link crosses the surface. The constant $1000 \text{ (kg/m}^3\text{)}$, the density of fresh water) was hard-coded directly into the expression. After the fix, the same line reads

$$\vec{F}_b = \left(0, 0, -\rho_w(t, x, y, z) \frac{mg}{\rho} \min\left(\frac{h_s + r - z}{2r}, 1\right) \right), \quad (18)$$

where $\rho_w(t, x, y, z)$ comes from a density callback threaded in from the arena’s water configuration — for the constant-density case, a single value read from the experiment’s YAML. The diff is four lines: a `water_density` parameter on the buoyancy function’s signature, and the literal 1000 replaced by it at the call site. The change is small, but the assumption it removes was not: the hard-coded constant silently made every simulated environment fresh water at standard conditions, whatever the configuration file said. Density now follows the same callback pattern already used for surface height, viscosity, and velocity in `WaterPropertiesExtension`.

6 Tetrahedron-Method Buoyancy: Centre of Buoyancy from Collision Geometry

6.1 The Problem: Coincident Centre of Buoyancy and Centre of Mass

The pre-existing `compute_buoyancy` routine (Eqs. (17)–(18)) is not a spatially-resolved computation at all. The submerged fraction is a linear ramp in depth, $\min((h_s + r - z)/2r, 1)$, and the resulting force attaches at one fixed point: the link’s bounding-sphere centroid, which coincides with its centre of mass. Code and version history both confirm this was the only buoyancy model present before the work described here.

The model has no notion of *where* on the body buoyancy acts relative to where the mass sits, and that is the defect. Buoyancy torque is

$$\vec{\tau}_b = (\vec{r}_{\text{cob}} - \vec{r}_{\text{com}}) \times \vec{F}_b,$$

so placing $\vec{r}_{\text{cob}}$ on $\vec{r}_{\text{com}}$ sets the lever arm, and hence the torque, to zero by construction. A real partially-submerged body feels a genuine righting or capsizing moment whenever the centroid of its *submerged volume* is offset from its centre of mass, which for the segmented, asymmetric ZBot body it always is. The two models therefore agree on where a body floats and disagree entirely on whether it stays upright — which is what makes the metacentric check of Section 6.4 the test that separates them.

6.2 The Fix: Per-Primitive Tetrahedron Decomposition of Submerged Volume

The fix computes the true centre of buoyancy of each rigid body directly from its own collision geometry, rather than from a single bounding-sphere proxy. For hydrodynamics purposes, every body in the model is a small set of collision *primitives*: boxes, spheres, cylinders, capsules. The method proceeds in three stages. First, turn each primitive into a closed, watertight triangular surface mesh, once, when the model loads. Second, at every step that needs an exact answer, clip that mesh against the water plane and decompose the submerged region into tetrahedra. Third, combine the tetrahedra into a submerged volume and a centre of buoyancy.

6.2.1 Stage 1: From Primitive to Mesh

MuJoCo’s collision geometry gives each body a small set of primitives (box, sphere, cylinder, capsule), each described only by its analytic size parameters (half-extents, radius, half-length), with no triangle mesh of its own. Stages 2 and 3 need an explicit, closed, watertight triangle surface to clip against the water plane, so I built one by hand for each primitive type, once, in that primitive’s own local frame, centred on its own origin:

- **Box**, half-extents (h_x, h_y, h_z) : the 8 corners $(\pm h_x, \pm h_y, \pm h_z)$, each face split along one diagonal into two triangles, 12 in total. This mesh is the box exactly.
- **Sphere**, radius r : a standard UV triangulation with n_{lat} latitude rings and n_{lon} vertices

per ring (16×32 by default), a vertex at polar angle θ_i and azimuth ϕ_j sitting at

$$\vec{v}_{ij} = (r \sin \theta_i \cos \phi_j, r \sin \theta_i \sin \phi_j, r \cos \theta_i), \quad (19)$$

with the poles fanned into the first and last rings and each intermediate ring pair split into two triangles per quad. The result inscribes the sphere rather than matching it, so it underestimates the volume; Section 6.2.6 quantifies by how much.

- **Cylinder**, radius r , half-height h : two n_{seg} -vertex rings at $z = \pm h$ ($n_{\text{seg}} = 20$) joined by a quad strip for the side wall, with each flat cap fanned from an added centre vertex.
- **Capsule**, radius r , half-length ℓ (MuJoCo’s convention: total length $2\ell + 2r$): the same side wall between equator rings at $z = \pm \ell$, capped by two hemispheres from Eq. (19) restricted to $\theta \in [0, \pi/2]$ and offset by $\pm \ell$. The hemispheres reuse the equator ring’s own vertices, so the seam closes exactly.

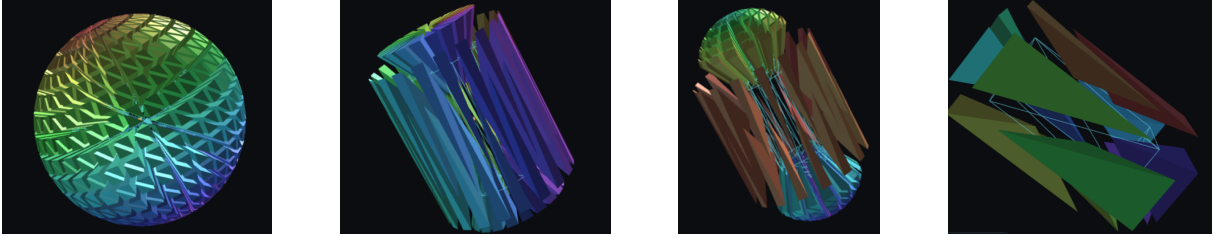


Figure 4: The four collision primitives, each decomposed into the tetrahedron fan of Stage 2. Left to right: sphere, cylinder, capsule, box. Individual tetrahedra are colour-coded, and the fan converges on the shared apex $\vec{p}$, taken on the water surface itself; the primitives are shown fully submerged, so that each closed mesh is fanned in its entirety. The sphere’s visible facets are the $n_{\text{lat}} \times n_{\text{lon}}$ tessellation whose volume error Section 6.2.6 quantifies.

Figure 4 shows the four primitive types with the tetrahedron fan of Stage 2 drawn on them. Each local mesh is built once, when the simulation model loads, and cached together with its own full (unclipped) volume, centroid, and bounding radius: the largest distance from that centroid to any of its vertices. I compute the full-mesh volume and centroid by running this same tetrahedron decomposition (Stages 2 and 3 below), but against an imaginary water surface placed above every vertex of the mesh, so every triangle is fully wet and the whole closed mesh is integrated exactly once, at load time. This cache is what Stages 2 and 3 operate on at every subsequent physics step; nothing about the primitive’s own geometry is recomputed at runtime.

6.2.2 Stage 2: Tetrahedron Decomposition of the Submerged Region

At a given step, each primitive’s cached local mesh transforms into world coordinates, using the body’s current position and orientation. An apex point $\vec{p}$ is then chosen on the water surface, horizontally at the mean position of the mesh’s *wet* vertices (falling back to all vertices when none is individually wet, purely to avoid a division by zero on a grazing primitive whose submerged volume is negligible anyway). This choice is free: any apex gives the same answer, for reasons Section 6.2.5 sets out, so the implementation picks whichever keeps the arithmetic best conditioned. Every triangle then contributes one or more tetrahedra with that shared apex, depending on how many of its three vertices lie at or below the water surface (Figure 5):

- **0 wet vertices**: the triangle is entirely above water and contributes nothing.
- **3 wet vertices**: the triangle is entirely submerged; it forms a single tetrahedron $(\vec{p}, \vec{a}, \vec{b}, \vec{c})$ with its own three vertices.

- **1 wet vertex:** only the wet vertex $\vec{v}_i$ and the two points where its adjoining edges cross the water surface, $\vec{p}_{ij}$ and $\vec{p}_{ik}$, are submerged; these three points form a single small wet triangle, itself the base of one tetrahedron ($\vec{p}, \vec{v}_i, \vec{p}_{ij}, \vec{p}_{ik}$).
- **2 wet vertices:** the submerged region is a quadrilateral bounded by the two wet vertices and the two surface-crossing points on the edges leading to the dry vertex; this quadrilateral is split into two triangles, each contributing its own tetrahedron.

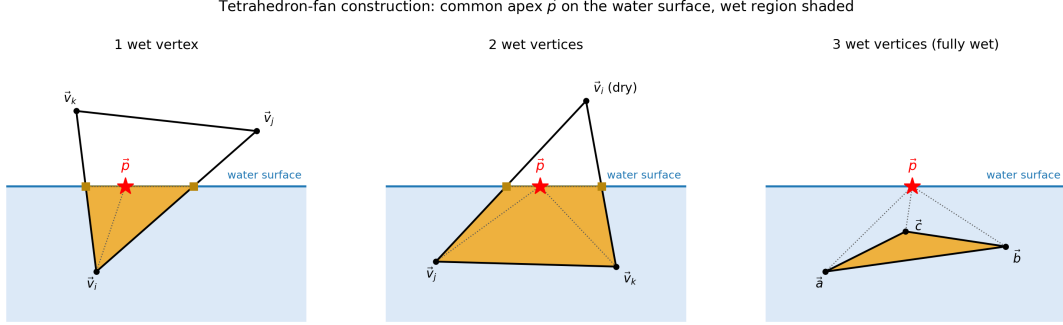


Figure 5: The three cases of Stage 2, all sharing the common apex $\vec{p}$, which lies on the water surface above the mean horizontal position of the wet vertices. The wet region is shaded and the surface-crossing points are marked with squares. Left to right: 1, 2 and 3 wet vertices.

The surface-crossing points in the 1- and 2-wet-vertex cases come from linear interpolation along the edge, using the signed depths d_i, d_j (positive below the surface, negative above) of its two endpoints:

$$t = \frac{d_i}{d_i - d_j}, \quad \vec{p}_{ij} = \vec{v}_i + t(\vec{v}_j - \vec{v}_i), \quad t \in [0, 1], \quad (20)$$

which is exact, because depth varies linearly along a straight edge: the point where it crosses zero is just the root of a straight line between two known values, found the same way.

6.2.3 Stage 3: Signed Volume, Weighted Centroid, and the Restored Torque

For a tetrahedron with apex $\vec{p}$ and base vertices $\vec{a}, \vec{b}, \vec{c}$, write the base vertices as vectors from the apex: $\vec{u} = \vec{a} - \vec{p}$, $\vec{v} = \vec{b} - \vec{p}$, $\vec{w} = \vec{c} - \vec{p}$. The signed volume is one-sixth of the scalar triple product of these three edge vectors,

$$V_{\Delta} = \frac{1}{6} \vec{u} \cdot (\vec{v} \times \vec{w}), \quad (21)$$

equivalently one-sixth of the determinant of the 3×3 matrix whose rows are $\vec{u}, \vec{v}, \vec{w}$.

The delivered implementation evaluates this in Cython (`buoyancy_cy.pyx`), with the edge crossing, the signed volume and the centroid accumulation all typed scalar routines, so the per-triangle loop never re-enters Python. It reproduces an earlier `numpy` version term for term, which is what lets that version stay in the tests as a reference implementation. Either way the cost per primitive scales with triangle count, which is why Stage 1's mesh resolution and the early rejection below matter as much as they do.

The sign of V_{Δ} follows the vertex winding, which Stage 1 fixes outward by construction (Section 6.2.4). Summing signed volumes over every tetrahedron, fully-wet and clipped cases together, therefore gives the true submerged volume without treating any case specially:

$$V = \sum_k V_{\Delta, k}. \quad (22)$$

Each tetrahedron's own centroid is the simple mean of its four vertices:

$$\vec{c}_\Delta = \frac{1}{4}(\vec{p} + \vec{a} + \vec{b} + \vec{c}), \quad (23)$$

The centre of buoyancy of the whole submerged shape is the volume-weighted average of these per-tetrahedron centroids:

$$\vec{r}_{\text{cob}} = \frac{\sum_k V_{\Delta,k} \vec{c}_{\Delta,k}}{\sum_k V_{\Delta,k}} = \frac{1}{V} \sum_k V_{\Delta,k} \vec{c}_{\Delta,k}. \quad (24)$$

Eq. (24) is what makes an irregular, partially submerged shape tractable: each tetrahedron integrates exactly, and the decomposition follows whatever shape the waterline happens to cut across the mesh.

Once the true, off-centre $\vec{r}_{\text{cob}}$ is available, I apply the buoyancy force there instead of at the centre of mass. The restoring torque is now computed correctly:

$$\vec{\tau}_b = (\vec{r}_{\text{cob}} - \vec{r}_{\text{com}}) \times \vec{F}_b, \quad \vec{F}_b = (0, 0, -\rho_w g V), \quad (25)$$

which is nonzero whenever the submerged-volume centroid and the mass centroid genuinely differ, restoring the physically correct righting behaviour that Eq. (17)'s single-point model could not produce. Figure 6 shows the full robot floating under this model.

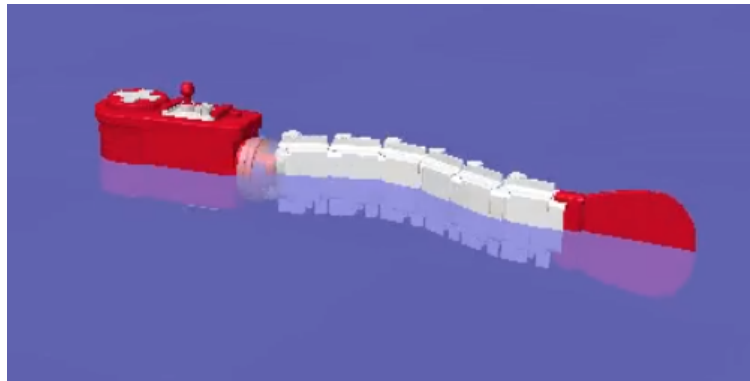


Figure 6: ZBot at the water surface in the FARMS simulation, with buoyancy computed per collision primitive by the tetrahedron method and applied at each body's own centre of buoyancy.

Two assumptions carry that construction, and both need justifying before its results can be trusted: that the signed-volume sum really recovers the enclosed volume, which holds only if every triangle is wound consistently outward, and that fanning an *open*, waterline-clipped sheet to an arbitrary apex is legitimate at all. The next two subsections take them in turn.

6.2.4 The Winding-Order Convention

Stage 3's signed-volume sum (Eq. (22)) only recovers the enclosed volume if every triangle's implied normal points outward. This matters more than it looks: a single triangle wound the wrong way *subtracts* its tetrahedron instead of adding it, and among many triangles that corrupts the total without producing an obviously wrong answer such as a negative volume.

I use the usual right-hand convention, in which the ordered vertices $(\vec{a}, \vec{b}, \vec{c})$ imply the face normal

$$\vec{n} = (\vec{b} - \vec{a}) \times (\vec{c} - \vec{a}), \quad (26)$$

by the right-hand rule: curl the fingers from $\vec{a} \rightarrow \vec{b} \rightarrow \vec{c}$, and the thumb points along $\vec{n}$. For a primitive centred on its own local origin — and all four here are star-shaped about their centre — a triangle is correctly wound exactly when $\vec{n}$ points away from that origin, i.e. when

$\vec{n} \cdot (\vec{a} + \vec{b} + \vec{c}) > 0$. Every triangle emitted in Stage 1 was checked against this test when its generator was written: the box's six faces individually, one mid-strip quad and one pole fan of the sphere (whose indexing pattern all the rest reuse), and the shared side-wall and cap patterns of the cylinder and capsule. Winding is therefore fixed at code-writing time, and Stages 2 and 3 assume outward normals at every physics step with no runtime re-verification. The box's exact volume in Table 1 is the end-to-end confirmation that the assumption holds.

Worked example: the box's bottom face. The face at $z = -h_z$ has corners $\vec{v}_0 = (-h_x, -h_y, -h_z)$, $\vec{v}_1 = (h_x, -h_y, -h_z)$, $\vec{v}_2 = (h_x, h_y, -h_z)$, $\vec{v}_3 = (-h_x, h_y, -h_z)$, split into triangles $(\vec{v}_0, \vec{v}_3, \vec{v}_2)$ and $(\vec{v}_0, \vec{v}_2, \vec{v}_1)$. Applying Eq. (26) to the first,

$$\vec{n} = (\vec{v}_3 - \vec{v}_0) \times (\vec{v}_2 - \vec{v}_0) = (0, 2h_y, 0) \times (2h_x, 2h_y, 0) = (0, 0, -4h_x h_y), \quad (27)$$

which points along $-\hat{z}$: the correct outward direction for a bottom face, for any positive h_x, h_y . The winding chosen for each face is simply whichever diagonal split keeps this sign right.

6.2.5 Correctness of the Common-Apex Fan Construction

Computing an enclosed volume by fanning a triangulated surface to a common apex is a standard technique in computational geometry, used for polyhedral mass properties [7]; what needs justifying here is its use on a surface that the waterline has left open. The clipped, wet-only set of triangles from Stage 2 is not itself a closed surface. Discarding the dry triangles and cutting the straddling ones at the waterline leaves an open sheet, bounded by a ragged loop of surface-crossing points along the waterline. Computing an enclosed volume from an open sheet is not well posed on its own; properly closing it would mean triangulating a flat “cap” polygon spanning that waterline loop and adding its contribution too. The tetrahedron-fan construction never does this explicitly. The reason it does not need to is a direct consequence of the divergence theorem.

For any closed surface S enclosing a volume V , and any fixed reference point $\vec{p}$, not necessarily inside or on S ,

$$V = \frac{1}{3} \oint_S (\vec{r} - \vec{p}) \cdot \hat{n} dA, \quad (28)$$

Here $\vec{r}$ ranges over points on S and $\hat{n}$ is the outward unit normal at each point. This follows from applying the divergence theorem to the vector field $\vec{F}(\vec{r}) = \frac{1}{3}(\vec{r} - \vec{p})$, whose divergence is $\nabla \cdot \vec{F} = 1$ everywhere: $\oint_S \vec{F} \cdot \hat{n} dA = \int_V \nabla \cdot \vec{F} dV = V$. Crucially, Eq. (28) holds for *every* choice of $\vec{p}$. Subtracting the constant field $\frac{1}{3}\vec{p}$ contributes $-\frac{1}{3}\vec{p} \cdot \oint_S \hat{n} dA$, and $\oint_S \hat{n} dA = 0$ for any closed surface, again by the divergence theorem applied to a constant vector field. This term vanishes identically, regardless of where $\vec{p}$ lies, which is what licenses the use of an arbitrary common apex.

On a single flat triangle $(\vec{a}, \vec{b}, \vec{c})$ the integrand $(\vec{r} - \vec{p}) \cdot \hat{n}$ is constant, so its contribution to Eq. (28) is that constant times the triangle's area — which is exactly the tetrahedron volume of Eq. (21). Summing over a closed triangulated surface therefore reproduces Eq. (28) term by term, and that is the identity Eqs. (21)–(22) implement.

What remains is the *open* sheet. Close it with any capping surface S_{cap} spanning the waterline loop; the enclosed volume depends only on that shared loop, not on the cap chosen, since two such closed surfaces differ by a closed surface enclosing zero volume. Fanning every wet triangle to a shared apex $\vec{p}$ is exactly the choice $S_{\text{cap}} =$ the cone from $\vec{p}$ to the waterline loop, whose own triangles contribute a volume that cancels from the total. The algorithm therefore integrates the clipped region correctly without ever triangulating the water plane, and the apex may go anywhere.

That freedom is not merely formal: supplying the same mesh with an apex above all of its vertices, or displaced several metres horizontally, reproduces the volume and centre of buoyancy to floating-point round-off. Placing the apex on the waterline above the submerged region serves only to keep the coordinates in Eq. (21) small and the arithmetic well-conditioned.

6.2.6 Mesh Discretization Error

A curved primitive’s mesh only approximates its true surface, and a polyhedron inscribed in a convex body always underestimates its volume. To quantify this I ran the same tetrahedron-decomposition code used at runtime against each primitive’s own vertices, with a water plane placed above every vertex so the whole closed mesh integrates at once, and compared the result to the exact analytic volume (Table 1).

Primitive	Default resolution	Mesh volume error
Box	exact by construction	$< 10^{-14}$
Sphere	$n_{\text{lat}} = 16, n_{\text{lon}} = 32$	1.60%
Cylinder	$n_{\text{seg}} = 20$	1.64%
Capsule	$n_{\text{lat}} = 6, n_{\text{lon}} = 16$ (at $r = \ell$)	3.21%

Table 1: Volume error of each primitive’s cached mesh against its exact analytic volume, at the default resolutions in `primitive_meshes.py`. Measured by running the delivered tetrahedron decomposition on the delivered mesh generator. The sphere figure reproduces the “ $\sim 1.6\%$ sphere-volume error” the module’s own docstring quotes for these defaults. Spheres do not normally incur this error at all: the default `analytic` method solves them in closed form (Section 6.2.7), and the mesh path applies only to shapes with no closed-form solution, or when `cob_method: mesh` is forced.

The box is integrated exactly, to floating-point round-off, which is a useful check in its own right: it confirms both the winding convention of Section 6.2.4 and the signed-volume sum of Eq. (22) on a case with a known closed-form answer. The capsule is the worst of the four, at 3.21%, its hemispherical caps being the most coarsely tessellated part of any primitive.

These resolutions were selected rather than inherited. The sphere was originally tessellated at 32×64 and was coarsened deliberately to cut the per-step clipping cost, which is proportional to face count. That trades a fourfold reduction in faces, 3968 down to 960, for a volume error rising from 0.40% to the 1.60% in the table — a sensible operating point for meshes whose only purpose is to integrate a volume and which are never rendered. The resolutions remain configurable through `cob_options.py` on the arena’s water configuration, so the table records defaults rather than fixed costs: an intermediate 24×48 sits at 0.71%.

The full-mesh figures do, however, understate the error that matters at runtime, because the hot path clips against the waterline rather than integrating a whole mesh. Measuring the clipped volume against the exact spherical cap across the range of submergence, at the default 16×32 , the error runs from 1.50% at nine-tenths submerged to 3.52% at one-eighth: a shallow cap is resolved by proportionally fewer triangles. The centre of buoyancy is far better behaved than the volume, agreeing with the analytic cap centroid to within 0.06 mm on a 50 mm sphere across that whole range, which is why settling heights come out as accurate as Section 6.3 shows even where the volume error is a few percent.

6.2.7 Closed-Form Shapes

Where a shape admits a closed-form submerged volume, tessellation is unnecessary, and `analytic_shapes.py` solves it directly. The sphere is the case implemented: the submerged region of a partially-immersed sphere is a spherical cap, whose volume and centroid are

$$V_{\text{cap}}(h) = \frac{\pi h^2(3r - h)}{3}, \quad d = \frac{3(2r - h)^2}{4(3r - h)}, \quad (29)$$

with h the cap height and d the centroid’s depth below the sphere’s own centre (at $h = r$ this gives $d = 3r/8$, the standard hemisphere result). Evaluating these is $O(1)$ and carries no

tessellation error whatever, so a sphere never touches the mesh path and none of Table 1’s sphere error applies to it.

The sphere was implemented first because it is the simplest case and the one the framework’s other users are most likely to have — but it is also the one shape ZBot does not use. A box and a cylinder both admit closed-form submerged volumes under an arbitrary waterline too, and extending the analytic path to them would remove the mesh from this robot’s collision set entirely. Until that is done, ZBot’s own geometry stays on the tessellated path and carries the errors quantified above.

The distinction is material, as the ball-drop validation of Section 6.3 demonstrates. Solving the equilibrium for a 10 cm ball at $\rho = 800$ gives a closed-form settled centre of -21.286 mm, while the mesh path predicts at least half a millimetre deeper at every resolution tested. The stored FARMS run settles within a micrometre of the closed-form value, so the validation data were produced by the analytic path.

6.3 Validation Against Closed-Form Settling Theory

I validated the fix with a suite of ball- and cube-drop checks, sweeping density across two object sizes. Each spawns a sphere or cube of known size and density above a water surface at $z = 0$, with the tank floor at $z = -0.3$ m, and lets it settle under gravity, buoyancy and drag alone. Figure 7 shows a settled density sweep.



Figure 7: Density sweeps at equilibrium under the tetrahedron method. Left: spheres of increasing density, each settling to its own draft. Right: the same sweep for cubes, which additionally settle into a definite *attitude*, because an off-centre centre of buoyancy exerts a real moment. Several rest edge-down rather than flat; Section 6.4 shows that this is correct behaviour, not an artifact.

For a cube of side a and density $\rho < \rho_w$, the closed-form settling height follows directly from Archimedes’ principle, $\rho_w g V_{\text{sub}} = \rho g V$: a submerged depth $h_{\text{sub}} = (\rho/\rho_w) a$ and an equilibrium centre height, relative to the water surface,

$$Z_{\text{cube}} = \frac{a}{2} - h_{\text{sub}} = a \left(\frac{1}{2} - \frac{\rho}{\rho_w} \right). \quad (30)$$

For a sphere of radius r the submerged volume is a spherical cap, not linear in depth, so the equilibrium draft $y = h/r$ satisfies the cubic

$$y^3 - 3y^2 + \frac{4\rho}{\rho_w} = 0, \quad Z_{\text{sphere}} = r(1 - y), \quad (31)$$

solved for the real root $y \in [0, 2]$. A sinking object ($\rho \geq \rho_w$) instead rests at $Z = Z_{\text{floor}} + a/2$ or $Z_{\text{floor}} + r$.

6.3.1 Steady-State Error

The quantity worth comparing is the *settled* height, so a case enters the comparison only once it has stopped moving: I take the mean height over the last two half-second windows and require them to agree within 1 mm. Averaging over windows rather than reading endpoints makes the

test insensitive to where in an oscillation cycle the run happens to stop. Cases that fail are excluded rather than quoted, since a mean over a body still in motion is not an equilibrium.

One subtlety decides what to compare the cubes against. Section 6.4 shows a uniform cube is metacentrically unstable flat for $211.3 < \rho < 788.7 \text{ kg/m}^3$, so in that band it settles edge-down and Eq. (30) is the wrong reference. For an edge-down cube the submerged cross-section is a diamond of half-diagonal $a/\sqrt{2}$, giving

$$Z_{\text{edge}} = \begin{cases} a \left(\frac{1}{\sqrt{2}} - \sqrt{\rho/\rho_w} \right), & \rho/\rho_w \leq \frac{1}{2}, \\ a \left(\sqrt{1 - \rho/\rho_w} - \frac{1}{\sqrt{2}} \right), & \rho/\rho_w > \frac{1}{2}. \end{cases} \quad (32)$$

Each cube is therefore compared against whichever attitude the metacentric criterion predicts it will adopt. Table 2 collects the result for every case that reaches a steady state.

Shape	ρ (kg/m ³)	Attitude	Analytic (mm)	Simulated (mm)	Error (mm)
Sphere, 10 cm	500	—	−0.000	−0.000	−0.000
Sphere, 10 cm	800	—	−21.286	−21.228	+0.058
Sphere, 10 cm	1100	on floor	−250.000	−250.040	−0.040
Sphere, 30 cm	1100	on floor	−150.000	−150.040	−0.040
Cube, 10 cm	300	edge-down	+15.938	+15.681	−0.257
Cube, 10 cm	400	edge-down	+7.465	+7.720	+0.255
Cube, 10 cm	800	flat	−30.000	−30.085	−0.085
Cube, 10 cm	1100	on floor	−250.000	−250.010	−0.010
Cube, 10 cm	1200	on floor	−250.000	−250.018	−0.018
Cube, 30 cm	1000	on floor	−150.000	−150.000	−0.000
Cube, 30 cm	1100	on floor	−150.000	−150.010	−0.010
Cube, 30 cm	1200	on floor	−150.000	−150.018	−0.018

Table 2: Steady-state settling error of the tetrahedron method, over every logged case that reaches a steady state. Cubes in the metacentrically unstable band are compared against the edge-down equilibrium of Eq. (32), which is the attitude they actually adopt. The largest error anywhere in the set is 0.257 mm.

The largest steady-state error across the set is 0.257 mm, and for the spheres it is 0.058 mm (Section 6.2.7). The two edge-down cubes are a cross-check the method was never tuned for: against the flat-attitude formula they appear 4.3 and 2.3 mm wrong, against the edge-down equilibrium 0.26 mm. The settling data therefore reproduces, independently, the attitude that Section 6.4 predicts from the torque alone.

Excluded, and why: the neutrally-buoyant 10 cm cube at $\rho = 1000$ has no restoring force at all — at $\rho = \rho_w$ the body is in indifferent equilibrium at any depth — so it never settles, which is physics rather than a defect. (The 30 cm cube at the same density does appear in the table only because it is as tall as the tank is deep, and so comes to rest on the floor.) The 30 cm floating objects and the 500 kg/m³ cube are still oscillating when their runs end, with excursions up to 97 mm, because the drag model removes too little energy for them to converge within the logged window. No steady-state error exists for those cases and none is quoted. That under-damping is a drag-coefficient limitation, not a buoyancy one, and it is why the fluid model should be trusted for steady floating attitude rather than for settling dynamics.

6.4 Validation of the Righting Torque Against Metacentric Theory

The settling checks above test the buoyancy *force*, which the model this work replaces already got right (Section 6.1). They say nothing about the torque, which is the whole point of the fix. The sharpest test available is classical hydrostatic stability, which predicts not just a magnitude but a *sign change* at a precisely known location.

For a floating body, classical ship hydrostatics [8] gives the metacentric height

$$GM = KB + BM - KG, \quad BM = \frac{I_{wp}}{V_{sub}}, \quad (33)$$

decides small-angle stability: $GM > 0$ gives a restoring moment, $GM < 0$ a destabilizing one. Here I_{wp} is the second moment of the waterplane area and KB , KG are the heights of the centre of buoyancy and centre of mass above the keel. For a uniform cube of side a and density ρ floating flat, the draft is $T = (\rho/\rho_w)a$ and Eq. (33) reduces to

$$GM = \frac{T}{2} + \frac{a^2}{12T} - \frac{a}{2}, \quad (34)$$

which vanishes when $6u^2 - 6u + 1 = 0$ for $u = T/a$, i.e. at $\rho/\rho_w = (3 \pm \sqrt{3})/6$. So a uniform cube is predicted to be stable flat only for $\rho < 211.32$ or $\rho > 788.68 \text{ kg/m}^3$, and to capsize toward an edge-down attitude in between — a demanding target, since near the upper root the whole stability margin is a fraction of a millimetre.

I tested this directly, holding a 0.10 m cube at its free-floating draft, rolling it, and reading off the buoyancy torque, using the pure-Python reference implementation rather than the shipped Cython that reproduces it term for term (Figure 8). Three results follow. The submerged volume is exactly $\frac{1}{2}a^3$ at $\rho = 500$ for every roll angle, so Archimedes holds throughout the rotation and not just at the flat attitude. The torque is antisymmetric about 45° with a zero there, recovering the edge-down equilibrium. And bisecting on the small-angle torque puts the two sign changes at $\rho = 211.50$ and 788.50 kg/m^3 against Eq. (34)'s 211.32 and 788.68 — agreement to within 0.2 kg/m^3 on a quantity never supplied to the method.

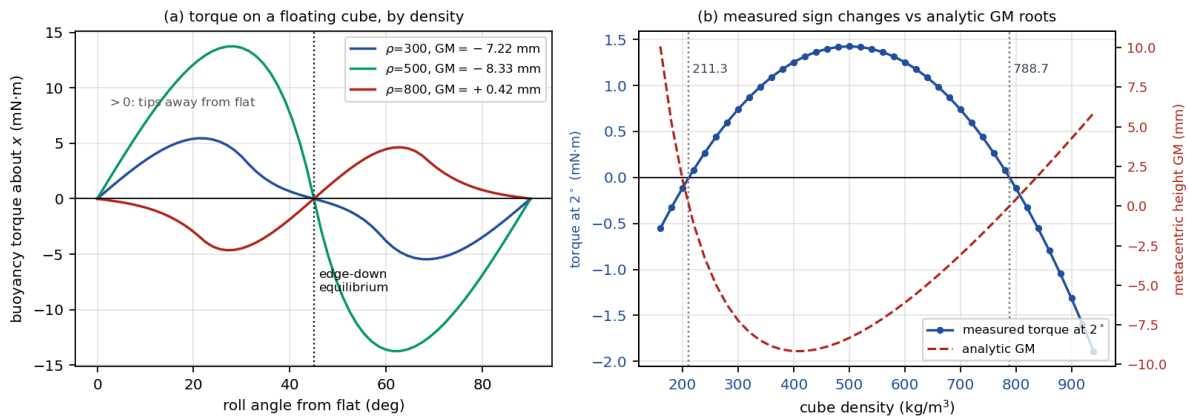


Figure 8: The tetrahedron method reproduces classical metacentric stability. (a) Buoyancy torque on a uniform floating cube against roll angle: at $\rho = 300$ and 500 kg/m^3 the flat attitude is destabilized and the cube is driven to the edge-down equilibrium at 45° , while at $\rho = 800$ the torque restores. (b) The measured small-angle torque changes sign at 211.50 and 788.50 kg/m^3 ; Eq. (34) puts the metacentric roots at 211.32 and 788.68 . This also explains the settled attitudes in Figure 7: the cubes there are not resting flat because, at those densities, flat is genuinely unstable.

The third of these results is the substantive one. Recovering the correct sign at $\rho = 800$, where Eq. (34) gives $GM = +0.42 \text{ mm}$, establishes that the method resolves a righting moment arising from a centre-of-buoyancy offset well below one millimetre. This is precisely the regime that the single-point model of Section 6.1 cannot represent, since that model sets the offset to zero by construction.

Because that conclusion rests on the same code it is testing, I cross-checked it against an independent method: uniform Monte-Carlo sampling of the rotated cube, counting points below

the waterline and averaging their positions. Over 0° to 45° of roll and 4×10^6 samples, the two agree on submerged volume to 0.1% and on centre-of-buoyancy offset to 0.01 mm.

6.5 The Method on the Robot Itself

The validation so far uses spheres and cubes, because those have closed-form answers to check against. ZBot — a chain of eight links carrying 21 boxes and 7 cylinders — has none, so the last question is whether the method does anything useful on its geometry.

Solving for the robot’s floating equilibrium from the delivered code gives a submerged volume of 2.95 L against a total collision volume of 3.29 L, so the robot floats almost awash, with about nine-tenths of it under water. Rolling the whole body about its long axis and recomputing at each angle gives the righting moment in Figure 9. It is restoring at every angle, rises monotonically, and reaches half a newton-metre by 60° .

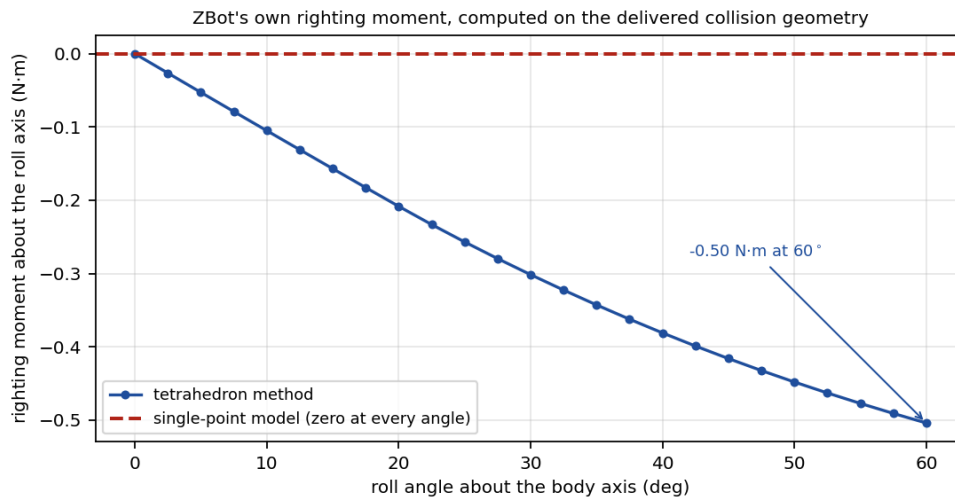


Figure 9: Righting moment on the assembled robot, computed on its own collision geometry at its own floating equilibrium. The moment opposes the roll at every angle and grows monotonically with it. The single-point model this work replaces returns exactly zero here, at every angle.

The comparison is the whole point of the section: the pre-existing model does not produce a smaller moment on this geometry, it produces none. A ZBot simulated under it floats at the right depth and rotates freely about its own axis — not a small discrepancy in a robot whose gait depends on staying upright while it beats its tail.

One caveat: this is a hydrostatic calculation on the delivered geometry, not a dynamic simulation of the robot righting itself. Logging a full dynamic swimming run under the corrected method is an obvious next step.

6.6 Cost, and How It Is Kept Inside the Real-Time Budget

The exact per-triangle clipping of Stage 2 runs once per collision primitive per link, at a cost proportional to that primitive’s face count. Nothing about the method is therefore inherently real-time: the budget is met by keeping the face count low, and a collision set described by a fine triangle mesh rather than by MuJoCo primitives would not fit inside it. Even with ZBot’s primitives, running the clip on every physics substep would dominate the simulation, so the delivered implementation does not. Four things keep the cost down, and the last two are what actually make the difference.

- **Mesh caching at load time** (Stage 1): each primitive’s local-frame mesh, full volume, centroid and bounding radius are computed once, when the model loads.

- **Bounding-sphere early rejection:** a primitive whose cached bounding sphere lies entirely above or below the water surface returns its cached zero or full volume immediately. The clipping loop runs only for primitives genuinely straddling the surface.
- **Closed-form shapes:** a sphere is solved by Eq. (29) in $O(1)$, with no clipping at all (Section 6.2.7).
- **Anchor-and-interpolate decimation:** under `analytic_fast` the exact computation runs once every `interp_steps` steps, with linear interpolation between anchors on the steps in between.

6.6.1 The Four Selectable Methods

`cob_options.py` exposes the choice on the arena’s water configuration, in either a flat form (`cob_method`, `cob_interp_steps`, ...) or a nested `cob:` block. `ramp` is the original single-point model of Section 6.1, retained as a fallback. `mesh` clips every primitive regardless of shape. `analytic` is exact every step and uses the closed form wherever one exists; it is the option’s default value. `analytic_fast` adds the decimation, with `interp_steps` defaulting to 20, and is what the delivered experiment configurations actually select — so it, not plain `analytic`, is the method the profiling and validation below exercise.

The decimation is a genuine approximation and worth stating precisely. Interpolating between the anchor at step n and the one at step $n - N$ means both endpoints are already in the past when step $n + k$ queries them, so the output tracks the true value with a lag of one anchor interval: N extension calls, which at the default $N = 20$ and the experiment’s two calls per 2 ms physics step (Section 3) is 20 ms, or 40 ms were the extension invoked once per step instead. That is acceptable for a slowly-bobbing float and not for fast transients, which is why `analytic` remains available as the exact, undecimated alternative, and why the configuration documents the accuracy trade-off explicitly rather than burying it.

6.6.2 Measured Cost

Profiling the swimming extension on the standard experiment configuration (5001 iterations at a 2 ms timestep, so 10.0 s simulated) gives Table 3. These are single runs on one desktop machine rather than averages over repeats, so the ratios between the two columns are more meaningful than the absolute times, and the real-time factor should be read as indicative of that machine.

Metric	<code>analytic_fast</code>	<code>ramp</code>
Total wall-clock time	3.118 s	2.421 s
Time inside the swimming extension	1.109 s	0.206 s
Per physics step, total	623 μ s	484 μ s
Per physics step, extension only	222 μ s	41 μ s
Fraction of the 2 ms step budget	31%	24%
Speed relative to real time	3.2 $\times$ faster	4.1 $\times$ faster

Table 3: Profiled cost of the delivered `analytic_fast` centre-of-buoyancy method against the pre-fix `ramp` fallback, same scenario and run length. Both run comfortably faster than real time; the exact method costs $\sim 29\%$ more wall-clock overall, and about $5.4\times$ more inside the extension itself.

These figures admit two readings, both correct, which should not be conflated. Taken overall, the exact method costs 29% more wall-clock time than the cheapest available fallback, a modest total overhead, and the simulation still runs $3.2\times$ faster than real time. Taken in isolation, however, its hydrodynamics computation costs approximately $5.4\times$ that of `ramp`. Both statements

hold simultaneously because hydrodynamics accounts for only about one third of the total step cost.

The decimation factor is visible directly in these numbers. Driving the reference implementation over ZBot’s collision geometry at its floating equilibrium — almost awash, so most primitives straddle the waterline at once, the most expensive region of the cost curve — costs a few milliseconds per step undecimated. Divided by the default $N = 20$ that lands in the low hundreds of microseconds, against the $222\ \mu\text{s}$ the profile records. Two routes sharing no code, a cProfile run of the whole simulation and a stopwatch on the geometry kernel alone, therefore agree: without decimation this geometry would not fit the budget, and with it, it fits several times over.

7 Deliverables and Current Status

7.1 Delivered Artifacts

The deliverable of this internship is working simulation and control infrastructure, together with two physics contributions to the framework underneath it. Concretely, the following exist and run at the end of the internship:

1. **A MuJoCo model and simulation of ZBot** (Section 2), ported from the pre-existing Webots description: MJCF model, robot interface, simulation loop, controller and environment abstractions, and run scripts.
2. **A FARMS integration for ZBot** (Section 3), built from scratch where none previously existed: SDF animat and arena descriptions, experiment scaffolding, configuration files, and a validation harness, on a tree vendoring the four FARMS packages as submodules.
3. **A CPG-driven bout-and-glide controller** (Section 4), `ZBotCPGController`, mirroring the physical robot’s C++ hardware controller class-for-class inside the FARMS controller interface, with both `BOUT_AND_GLIDE` and `CONTINUOUS` modes.
4. **An upstream fix to `farms_mujoco`** (Section 5), making water density configurable rather than hard-coded, merged upstream as pull request #43.
5. **A tetrahedron-decomposition centre-of-buoyancy method** (Section 6), with load-time mesh caching, bounding-sphere early rejection, a closed-form solution for spheres and an anchor-and-interpolate schedule, implemented in Cython, plus the ball- and cube-drop validation suite used to check it against closed-form settling theory. It is geometrically correct and runs inside its real-time budget under the `analytic_fast` method the delivered experiments select (Section 6.6). Unlike the density fix, this is not upstream: it lives on the `hydrodynamics` branch of a `farms_mujoco` fork, which is the branch the ZBot tree vendors as its submodule. Anyone pulling stock `farms_mujoco` gets the single-point ramp instead.
6. **A containerized, cross-platform deployment** (Section 3), with identical Linux and Windows Dockerfiles and the one unreconciled bind-mount asymmetry noted there.
7. **Documentation** for the FARMS stack and the ZBot integration, including the Docker-based setup, published as a documentation site (https://podalanga.github.io/farmsim_docs/) for the benefit of others in the laboratory picking this work up.

7.2 Scope Boundary and Next Steps

What was delivered is the infrastructure itself, not a swimming study conducted with it. The bout-and-glide versus continuous energy-efficiency comparison is the published contribution of Liu et al. [2], established experimentally on the physical robot, and reproducing it in simulation

was not part of the mandate (Section 1.3). This report accordingly presents no simulated cost-of-transport or swimming-speed figure.

The validation reported here is correspondingly targeted at the infrastructure: the traveling-wave kinematic check (Section 2), the ball- and cube-drop settling checks against closed-form theory (Section 6.3), the metacentric-stability check on the buoyancy torque and its Monte-Carlo cross-check (Section 6.4), the mesh-discretization measurements of Section 6.2.6, and the direct evaluation of the bout-gate state machine in Figures 2 and 3.

Three items are known to be unfinished, and should be read alongside that list rather than as footnotes to it. The bout gate’s smoothing weight has not been rescaled for the simulation’s control rate, so simulated tail amplitude is roughly twice the hardware’s (Section 4.4). The decimation that keeps the centre-of-buoyancy cost inside budget introduces a one-anchor-interval lag, tens of milliseconds at the default settings, which is fine for a slowly-bobbing float and not for fast transients (Section 6.6). And the drag model under-damps the settling transient badly enough that only steady-state floating attitude should currently be trusted (Section 6.3).

The natural next step is to run the `zbot_bout_glide` experiment in both modes and log joint targets alongside simulated velocity. Beyond exercising the whole stack end to end, that run would settle the joint-saturation question of Section 4.4 directly, since the logged joint targets are exactly the data that derivation predicts.

8 Known Issues and Future Work

8.1 The Inertia Tensor Problem

While porting the model, the inertia specification looked wrong: MuJoCo’s rendered inertia boxes did not line up with the geometry they represent. Tracing the model’s history shows the tensor passing through three states.

The earliest revision stored each body as a diagonal tensor plus a rotation carrying it into the body’s principal axes. A later manual rewrite re-expressed the same bodies in the unrotated body frame, as a full symmetric tensor; for the head,

$$\mathbf{I} = \begin{pmatrix} 8.56 & 0 & 0 \\ 0 & 2.98 & -0.263 \\ 0 & -0.263 & 7.68 \end{pmatrix} \times 10^{-3} \text{ kg} \cdot \text{m}^2. \quad (35)$$

Nothing is lost in that rewrite — it is the same physical tensor in a different frame, and the off-diagonal I_{yz} is precisely what the rotation had encoded. What it costs is legibility: a nonzero off-diagonal tilts the rendered inertia box while the body frame stays put, which is the symptom that prompted the investigation.

8.1.1 Recovering the Principal Frame

Diagonalizing Eq. (35) is worth doing, because the answer is more interesting than it first appears. Since $I_{xy} = I_{xz} = 0$, the x axis is already principal, and the problem reduces to the symmetric 2×2 block in the y – z plane. Its eigenvalues are 7.695 and $2.965 \times 10^{-3} \text{ kg} \cdot \text{m}^2$ — exactly the two values the model stores alongside 8.56×10^{-3} — and its principal axes lie at the angle that annihilates the off-diagonal term,

$$\tan 2\theta = \frac{2I_{yz}}{I_{yy} - I_{zz}} = 0.1119, \quad \theta = 3.19^\circ. \quad (36)$$

That angle is the point. The body’s principal axes are tilted only 3.19° off its geometric axes — a nearly-aligned frame, as one would expect for a roughly symmetric head. The earliest revision, however, stored the rotation as a quaternion equivalent to 93.19° about x , and reconstructing $\mathbf{R} \text{diag}(I_1, I_2, I_3) \mathbf{R}^\top$ from it reproduces Eq. (35) to within $1.3 \times 10^{-9} \text{ kg} \cdot \text{m}^2$. Both

are correct, and the 90° between them is not a discrepancy but a relabelling: MuJoCo stores `diaginertia` in a fixed order, and the extra quarter turn is what maps the moments onto the axes in that order. The frame barely moves; the labels swap.

Reading the 93.19° as a large physical misalignment would therefore be a mistake, and it changes what is actually wrong with the model today. The eigenvector lying 3.19° from the body y axis carries the *small* moment 2.965×10^{-3} , and the one 3.19° from z carries 7.695×10^{-3} .

8.2 Why the Current Model Is Still Wrong

A third pass, the URDF round-trip, produced the state the model is in now: every body carries a `diaginertia` with the correct principal moments and *no* `quat`. The rendered boxes line up again and the visual symptom is gone.

But the tensor is now a different tensor. Omitting `quat` does not mean the rotation is unknown; it means the rotation is the identity. Given the previous section, the consequence is specific and easy to state. The stored order puts 7.695×10^{-3} on the body y axis and 2.965×10^{-3} on z , whereas the principal axis lying nearest y actually carries the *small* moment and the one nearest z the large one. Dropping the rotation therefore does not introduce a 93° tilt — the frames were only 3.19° apart to begin with — it interchanges the moments about y and z . Since those differ by a factor of 2.6, the head is given roughly $2.6\times$ too much resistance to pitching and correspondingly too little to yawing, which is not a rounding-level error. The same omission propagates into the SDF that FARMS loads, whose `<inertia>` blocks carry these diagonal values with all products of inertia zero.

A visualization artifact was therefore traded for a dynamical error that does not show up visually. This can be resolved by writing `diaginertia` together with `quat` in the MJCF and the full symmetric tensor in the SDF, both of which the eigendecomposition already provides for every body.

8.3 The Case for System Identification

Diagonalizing an inertia tensor derived from an approximate CAD/URDF mass distribution only ever recovers the principal moments *of that approximate model*. It corrects a mathematical inconsistency in how the tensor is expressed, not the accuracy of the underlying mass/geometry estimate it came from. The real robot’s mass distribution (batteries, servomotors, wiring, the ballast tank shown in the hardware overview [2]) is not perfectly captured by any CAD-derived estimate. Only measuring the real hardware can establish the true principal axes and moments of the physical ZBot unit.

The project’s own version history independently supports this framing: a later commit on the same model traces the inertia values back to a Webots-version upgrade (R2018 to R2025) carried out by M. Louis Gevers during an earlier stage of the project, rather than to a first-principles CAD estimate produced for this report. The diagonalization of these matrices was performed as part of the present work during the port to MuJoCo, and re-verifying the underlying values against that earlier codebase remains an open task.

Two routes would settle it. A bifilar or trifilar pendulum measurement of each segment gives the moment about a known axis from the oscillation period, repeated about several axes to reconstruct the full tensor. Alternatively, a CAD estimate can be refined by matching simulated free response against the same motion measured on the hardware. Either determines the principal axes by measurement rather than inheriting them from a CAD export, and either must be repeated per segment, since the robot is not rigid overall. Until then, the model’s inertial parameters are not a validated physical property of the robot, and restoring the missing rotation only makes them internally consistent with the CAD estimate they came from.

8.4 Future Work

Directions follow from where this work stops, in rough order of how directly they build on it.

1. **Damp the settling transient.** The under-damped bobbing of Section 6.3 is a drag-coefficient problem, not a buoyancy one, and it is what currently stops the fluid model being trusted for anything but steady-state attitude. The Webots drag coefficients this project inherited are the obvious place to start.
2. **An ellipsoid-based centre of buoyancy and surface drag forces.** The sphere is currently the only shape with a closed-form solution (Section 6.2.7); every other primitive goes through mesh clipping and carries the tessellation error of Section 6.2.6, and ZBot's own collision set is boxes and cylinders, so none of it benefits. A partially-submerged ellipsoid has a closed-form submerged volume and centroid, as a sphere does, so an analytic path for ellipsoids would remove the mesh from a general shape class and would fit ZBot's elongated segments far better than either a sphere or the box-and-cylinder approximation used now. Furthermore, minimum bounding ellipsoids can be used in the future to compute surface drag forces more accurately.
3. **System identification of the inertial parameters.** The measurement campaign of Section 8.3, which is the only route to confirming the robot's true principal moments and axes rather than those of the CAD estimate.
4. **Global navigation.** With a working CPG and turning drive in place, the platform is ready for a navigation layer above the gait. Potential-field methods are the natural starting point, being cheap and straightforward to layer over the turning drive the controller already exposes. It waits on the real-time budget of Section 6.6: a planner has no headroom to run in until the buoyancy computation leaves some.

9 Conclusion

The internship set out to move ZBot from Webots onto the laboratory's standard simulation stack, and that objective was met. A researcher can now run ZBot under MuJoCo and FARMS, in either swimming mode, through the same experiment scaffolding as the laboratory's other animats. Section 7 lists what that stack consists of.

The larger technical contribution was unplanned. Carrying out the port exposed three defects in the simulation physics. Making water density configurable was a small fix with a clear boundary, and it is merged upstream, so every FARMS user benefits from it. Replacing the single-point buoyancy model was substantially larger: the pre-existing routine applied the buoyant force at a fixed proxy point, which produces no righting torque by construction, and correcting it required computing the true centre of buoyancy from each body's own collision geometry.

The most significant validation result is the metacentric check, because it tests the quantity the correction was made for rather than the one both models already get right. The method locates the two densities at which a uniform floating cube switches between stable and unstable to within 0.2 kg/m^3 of theory, which means it resolves righting moments from centre-of-buoyancy offsets of a few tenths of a millimetre. The model it replaces puts those offsets at zero by construction. Against closed-form settling theory its worst steady-state error across twelve cases is 0.257 mm.

Three limitations remain. First, the real-time margin depends on decimating the exact computation to one step in twenty, which buys the speed at the cost of a lag of tens of milliseconds in the buoyancy force. Second, the inherited drag coefficients remove too little energy for larger bodies to settle within a run, so the fluid model should be trusted for steady floating attitude

rather than settling dynamics. Third, the missing inertial rotation of Section 8.1 sits alongside these as a further small fix with a known form.

The infrastructure is a means rather than an end. The energy-efficiency question it serves has already been answered experimentally on the physical robot by Liu et al. [2]; what the stack adds is the ability to pose that class of question in simulation from here on. These limitations are documented rather than left implicit because the value of the infrastructure to whoever continues the project depends on knowing which of its parts have been validated, and which have only been implemented.

Acknowledgments

I thank Louis Gevers for his day-to-day guidance throughout the project and for patiently answering every question it raised; Javier Jia Jie Pey and Alexandre Guignard for explaining the CPG formalism this report builds on; and Prof. Auke Ijspeert for the opportunity to work on this project and for helping shape it toward what suited me. I am also grateful to Alessandro Crespi for his help arranging this internship and for providing the workstation this work was carried out on, and to Kohei Nishikawa, Dounia Hassar, and the rest of my colleagues at BIOROB, for their support throughout. I am particularly grateful to Jonathan Arreguit, a contributor to FARM-SIM, who created the initial ZBot FARMSIM repository this work was built on and who sat with me to guide me through using the framework. I really want to thank Alessandro Pazzaglia for bringing Churro, and Churro for being my emotional support. Finally, I thank my family in India, without whose support I would not have made it to Switzerland for this internship at all.

References

- [1] J. Arreguit, S. Tata Ramalingasetty, S. M. Danner, and A. J. Ijspeert. FARMS: Framework for Animal and Robot Modeling and Simulation. *bioRxiv* preprint 2023.09.25.559130, version 4, April 2026. <https://doi.org/10.1101/2023.09.25.559130>
- [2] X. Liu, F. A. Longchamp, L. Zunino, L. Gevers, L. R. Schneider, S. I. Bothner, A. Guignard, A. Crespi, G. Bellegarda, A. Bernardino, E. A. Naumann, and A. J. Ijspeert. Energy efficiency and neural control of continuous versus intermittent swimming in a fishlike robot. *Science Robotics*, 11(110):eadw7868, 2026. <https://doi.org/10.1126/scirobotics.adw7868>
- [3] A. J. Ijspeert, A. Crespi, D. Ryczko, and J.-M. Cabelguen. From swimming to walking with a salamander robot driven by a spinal cord model. *Science*, 315(5817):1416–1420, 2007. <https://doi.org/10.1126/science.1138353>
- [4] A. J. Ijspeert. Central pattern generators for locomotion control in animals and robots: A review. *Neural Networks*, 21(4):642–653, 2008.
- [5] D. Weihs. Energetic advantages of burst swimming of fish. *Journal of Theoretical Biology*, 48(1):215–229, 1974.
- [6] J. J. Videler and D. Weihs. Energetic advantages of burst-and-coast swimming of fish at high speeds. *Journal of Experimental Biology*, 97:169–178, 1982.
- [7] B. Mirtich. Fast and accurate computation of polyhedral mass properties. *Journal of Graphics Tools*, 1(2):31–50, 1996.
- [8] A. Biran and R. López-Pulido. *Ship Hydrostatics and Stability*. Butterworth-Heinemann, 2nd edition, 2013.

-
- [9] E. Todorov, T. Erez, and Y. Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5026–5033, 2012.